



Sorokin School

Модуль №3 задание "Parallel Fiction" [PDF]

Кодовое-название: Parallel Fiction: фабрика историй

Референс: *Pulp Fiction*

История предисловие к заданию:

[=] Редакция захлёбывается в черновиках: одни тексты «висят» на I/O, другие съедают CPU, а выпускающий орёт: «нужно вчера». Ваша миссия — запустить учебный конвейер на **CompletableFuture**: I/O (черновик) → CPU (редактура) → I/O (финализация) → сохранение.

За один запуск вы сравните **fixed thread pool** и **virtual threads**: сначала фиксированный пул, затем виртуальные потоки; между прогонами хранилище очищается. Ошибки редакции генерируются **с заданной вероятностью** и **локализируются** в конце цепочки — в хранилище попадает только успех. Домашка — полностью самостоятельный «атом», без внешних зависимостей.

Цель

- Собрать конвейер на **CompletableFuture** (supply/thenApply/thenAccept + exceptionally).
- Понять разницу **FixedThreadPool** vs **Virtual Threads** на I/O-heavy и коротких CPU-вставках (VT — не серебряная пуля).
- Научиться **локализовывать ошибки** в цепочках так, чтобы общий процесс не падал.
- Получить сравнение по времени для $n \in \{10, 100, 1000\}$ и сделать короткий вывод.

Правила и ограничения (единые)

- **Java 21.** Без сторонних зависимостей.
- Без CLI-флагов; все параметры как **константы** внутри `main`.
- Хранилище результатов — потокобезопасное (внутри `ConcurrentHashMap`).
- Ожидание — одним барьером через `CompletableFuture.allOf(...).join()`; **никаких** `get()/join()` посередине цепочек.

Архитектура и строгий API

1) `StoryTask` — входная неизменяемая задача

Назначение: перенос исходных данных.

Поля: `id: long`, `author: String`, `title: String`.

Фабрика:

```
1 public record StoryTask(long id, String author, String title) {
2     public static StoryTask of(long id, String author, String title) {
3         if (id <= 0 || author == null || author.isBlank() || title == null ||
4             title.isBlank()) {
5             throw new IllegalArgumentException("Invalid task");
6         }
7         return new StoryTask(id, author, title);
8     }
9 }
```

2) `StoryResult` — итог обработки

Назначение: фиксация финального результата.

Поля: `taskId: long`, `finalText: String`.

Фабрика:

```
1 public record StoryResult(long taskId, String finalText) {
2     public static StoryResult of(long taskId, String text) {
3         if (taskId <= 0 || text == null || text.isBlank()) {
4             throw new IllegalArgumentException("Invalid result");
5         }
6         return new StoryResult(taskId, text);
7     }
8 }
```

```
8 }
```

3) **StoryService** — шаги конвейера + задержки/нагрузки

Конструктор:

```
1 public final class StoryService {
2     // все ms ≥ 0; 0 < chanceOfEditingError ≤ 1 → иначе IllegalArgumentException
3     public StoryService(int fetchDraftMs, int editTextMs, int finalizeMs, double
        chanceOfEditingError) { ... }
4
5     // Имитация внешнего I/O (блокирующая задержка): вернёт "draft"
6     public String fetchDraft(StoryTask task) { ... }
7
8     // Имитация CPU (busy-loop); с заданной вероятностью бросает исключение
9     public String editText(String draft) { ... }
10
11    // Имитация I/O + формирование StoryResult
12    public StoryResult finalizeStory(StoryTask task, String editedText) { ... }
13 }
```

4) **StoryStorage** — потокобезопасное хранилище

Внутри: `ConcurrentHashMap<Long, StoryResult>`.

Конструктор: `(int saveTimeMillis)` — задержка I/O при сохранении; `saveTimeMillis > 0`.

Методы:

```
1 public final class StoryStorage {
2     public void save(StoryResult result) { ... }    // upsert по taskId, с
        I/O-задержкой
3     public Optional<StoryResult> get(long taskId) { ... }
4     public int size() { ... }
5     public Map<Long, StoryResult> snapshot() { ... } // немодифицируемая копия
6     public void clear() { ... }
7 }
```

5) **StoryPipelineDemo** — единый учебный `main`

Константы (пример дефолтов):

```
1 POOL_SIZE = 100;
2 IO_MILLIS = 100;           // I/O-задержка на fetch/finalize/save
3 CPU_MILLIS = 1;           // CPU-нагрузка на edit
4 CHANCE_OF_ERROR = 0.2;    // вероятность ошибки на editText (0 < chance ≤ 1)
```

Сценарий двух прогонов в одном процессе:

1. **Fixed Thread Pool** → запуск конвейера → фиксация времени и `storage.size()`.
2. `storage.clear()`.
3. **Virtual Threads** (`Executors.newVirtualThreadPerTaskExecutor()`) → повтор запуска → фиксация времени и `size()`.

Внутренний конвейер `runPipeline(...)`

- Для **каждой** задачи создайте **асинхронную цепочку** из четырёх шагов:
 1. асинхронно получить черновик (**I/O**, выполняется на переданном executor),
 2. асинхронно выполнить редактирование (**CPU**, можно оставить без явного executor, чтобы выполнить на общем `ForkJoinPool`),
 3. асинхронно финализировать (**I/O**, на переданном executor),
 4. сохранить результат в хранилище.
- В **конце цепочки** добавьте обработку ошибок так, чтобы **любая ошибка**:
 - логировалась,
 - **не** приводила к падению приложения,
 - **не** записывала результат в хранилище.
- Соберите все будущие результаты и **дождитесь завершения всего набора** через общий барьер (ожидание завершения всех цепочек).

Наблюдение для обсуждения: когда часть шагов запускается «без явного executor», они используют **общий ForkJoinPool**. Это наглядно показывает влияние **двух разных пулов** на конвейер.

Как запускать и что проверять

Класс входа: `StoryPipelineDemo.main(String[] args)`

1. **Данные.** Сгенерируйте `n` задач: `StoryTask.of(1..n, "Author-i", "Chapter-i")`, где $n \in \{10, 100, 1000\}$.

2. **Прогон #1 (Fixed).** Создайте `StoryStorage` , `StoryService` , `Executors.newFixedThreadPool(P00L_SIZE)` ; вызовите `runPipeline(...)` ; зафиксируйте время, `storage.size()` , распечатайте 2–3 результата из `snapshot()` .
3. **Очистка.** `storage.clear()` .
4. **Прогон #2 (Virtual Threads).** Создайте `Executors.newVirtualThreadPerTaskExecutor()` , вызовите `runPipeline(...)` ; снова зафиксируйте время и `storage.size()` .
5. **Сравнение.** Для всех `n` сравните времена Fixed vs VT и сделайте короткий вывод. Поиграйте параметрами `IO_MILLIS` / `CPU_MILLIS` / `CHANCE_OF_ERROR` и описывайте, как меняется пропускная способность/доля ошибок. (Помните: VT особенно полезны на I/O-bound нагрузке.)

«Проверь себя» (самопроверка)

- `storage.size()` $\approx$ число задач – число ошибок на `editText` (ошибки залогированы; результаты не сохранены).
- Логи показывают **асинхронный порядок**: шаги `fetch/edit/finalize` идут параллельно; имена потоков/VT меняются.
- Оба прогона завершились без зависаний; `storage.clear()` между ними реально обнуляет `size()` .
- Для `n = 10/100/1000` сделано **сравнение времени** Fixed vs VT и краткий вывод «где быстрее и почему».
- В цепочках **нет блокирующих** `get()/join()` — только финальный `allof(...).join()` .



Мини-чеклист (качество реализации)

- **Java 21**, всё конфигурируется константами в `main` .
- Фабрики `StoryTask.of(...)` и `StoryResult.of(...)` валидируют вход.
- `StoryStorage` потокобезопасен; `snapshot()` — немодифицируемая копия.
- Исключения в `editText` локализованы через `exceptionally` , общий процесс не падает.
- Между прогонами Fixed/VT — `storage.clear()` ; в выводе — различия по времени и наблюдения.
- Логирование минималистично и полезно (стадия + `taskId` + краткая причина ошибки).



Подсказки (без готовых решений)

- **Где мерить время?** Проще всего — снаружи `runPipeline(...)` :

```
1 long t0 = System.nanoTime();
2 runPipeline(exec, tasks, service, storage);
3 long ms = (System.nanoTime() - t0) / 1_000_000;
4 Logger.info("Elapsed: %d ms", ms);
```

- **Консистентность executors.** Хотите «чистый» эксперимент — передайте один и тот же `executor` во все `then*Async` и сравните результат с исходным.
- **I/O vs CPU.** Увеличение `IO_MILLIS` усиливает преимущество VT; рост `CPU_MILLIS` нивелирует разницу (цена планирования растёт).

Приложение А — Утилиты (можно использовать “как есть”)

Эти утилиты упоминаются в задании, но не являются частью решения конвейера. Их реализация дана для удобства запуска и замеров.

Logger — компактный потокобезопасный логгер

```
1 package dev.sorokin.utils;
2
3 /** Компактный logger: печатает имя потока и метку времени */
4 public final class Logger {
5     private static volatile boolean logEnabled = true;
6     private Logger() {}
7
8     public static void log(String fmt, Object... args) {
9         if (!logEnabled) return;
10        String message = (args == null || args.length == 0) ? fmt :
String.format(fmt, args);
11        Thread current = Thread.currentThread();
12        String logFormat = "[%tT.%1$tL][thread:%s] %s%n";
13        System.out.printf(logFormat, System.currentTimeMillis(),
current.getName(), message);
14    }
15    public static void info(String fmt, Object... args) { logWithLevel("INFO",
fmt, args); }
16    public static void error(String fmt, Object... args) {
logWithLevel("ERROR", fmt, args); }
17    public static void disablePrintLogs() { logEnabled = false; }
```

```

18     public static void enablePrintLogs() { logEnabled = true; }
19
20     private static void logWithLevel(String level, String fmt, Object... args)
21     {
22         String finalFmt = "[%s] %s".formatted(level.toUpperCase(), fmt);
23         log(finalFmt, args);
24     }

```

ThreadSleepUtil — безопасный сон (с джиттером)

```

1 package dev.sorokin.utils;
2
3 import java.security.SecureRandom;
4
5 /** Обёртка над Thread.sleep(...) */
6 public final class ThreadSleepUtil {
7     private static final SecureRandom RANDOM = new SecureRandom();
8     private ThreadSleepUtil() {}
9
10    /** Спит указанное время; при прерывании восстанавливает флаг. */
11    public static void safeSleepWithoutThrow(long ms) {
12        if (ms <= 0) return;
13        try { Thread.sleep(ms); } catch (InterruptedException e) {
14            Thread.currentThread().interrupt();
15        }
16
17        /** Спит указанное время с небольшим джиттером; при прерывании
18        восстанавливает флаг. */
19        public static void safeSleepWithJitter(long ms) {
20            if (ms <= 0) return;
21            try {
22                long jitter = RANDOM.nextLong(ms / 2);
23                if (RANDOM.nextBoolean()) jitter *= -1;
24                Thread.sleep(ms + jitter);
25            } catch (InterruptedException e) { Thread.currentThread().interrupt();
26        }
27    }
28
29    /** Спит случайное время в диапазоне; при прерывании восстанавливает флаг.
30    */
31    public static void safeSleepRandomMillis(long lowerBound, long upperBound)
32    {
33        try {
34            long sleepMillis = RANDOM.nextLong(lowerBound, upperBound);
35            Thread.sleep(sleepMillis);

```

```

31         } catch (InterruptedException e) { Thread.currentThread().interrupt();
32     }
33 }

```

BusyCpuUtil — искусственная CPU-нагрузка

```

1 package dev.sorokin.utils;
2
3 import java.security.SecureRandom;
4
5 /** Утилита активной загрузки CPU (busy-wait) на заданное время. */
6 public final class BusyCpuUtil {
7     private static final SecureRandom RANDOM = new SecureRandom();
8     private BusyCpuUtil() {}
9
10    /** Крутится на CPU указанное кол-во миллисекунд. */
11    public static void spinOnCpuMillis(long ms) {
12        long end = System.nanoTime() + ms * 1_000_000L;
13        long acc = 0;
14        while (System.nanoTime() < end) {
15            acc += (acc * 31 + 7); // немного бесполезной арифметики
16        }
17        if (acc == 42) System.out.print(""); // анти-оптимизация JIT
18    }
19
20    /** То же, но с небольшим джиттером. */
21    public static void spinOnCpuMillisWithJitter(long ms) {
22        if (ms <= 0) return;
23        long jitter = RANDOM.nextLong(ms / 2);
24        if (RANDOM.nextBoolean()) jitter *= -1;
25        spinOnCpuMillis(ms + jitter);
26    }
27 }

```

MeasurementUtils — удобные замеры (опционально)

```

1 package dev.sorokin.utils;
2
3 import java.util.function.Supplier;
4
5 /** Утилиты для измерений */
6 public final class MeasurementUtils {
7     private MeasurementUtils() {}

```



```
8
9  /** Измеряет и логирует длительность выполнения задачи (Supplier-версия)
   */
10 public static <T> T measure(String taskName, Supplier<T> task) {
11     long startNanos = System.nanoTime();
12     try {
13         return task.get();
14     } catch (Exception e) {
15         Logger.info("Exception in task-name: %s message:", taskName,
16             e.getMessage());
17         throw e;
18     } finally {
19         long endNanos = System.nanoTime();
20         long timeElapsedMs = (endNanos - startNanos) / 1_000_000;
21         Logger.info("Time: task-name: %s time: %sms", taskName,
22             timeElapsedMs);
23     }
24 }
25
26 /** Измеряет и логирует длительность выполнения задачи (Runnable-версия)
   */
27
28 public static void measure(String taskName, Runnable task) {
29     long startNanos = System.nanoTime();
30     try {
31         task.run();
32     } catch (Exception e) {
33         Logger.error("Exception in task-name: %s message: %s", taskName,
34             e.getMessage());
35         throw e;
36     } finally {
37         long endNanos = System.nanoTime();
38         long timeElapsedMs = (endNanos - startNanos) / 1_000_000;
39         Logger.info("Time: task-name: %s time: %sms", taskName,
40             timeElapsedMs);
41     }
42 }
```